

שאלה 1

א. כתבו פונקציה בשם `set_line_length` שמקבלת שם של קובץ קלט ומספר שלם חיובי n ומדפיסה את תוכן הקובץ ל-`standard output` כך שבכך שורה יהיו n תווים. החליפו כל $\backslash$ בקובץ בתו רווח. הוסיפו n אחרי כל n תווים.

למשל: עבור הקובץ `song.txt` שמכיל את הטקסט הבא:

```
Oh! Darling, please believe me
I'll never do you no harm
Believe me when I tell you
I'll never do you no harm
```

ועבור $n=15$ הפלט יהיה

```
Oh! Darling, pl
ease believe me
  I'll never do
you no harm Bel
ieve me when I
tell you I'll n
ever do you no
harm
```

לצורך מימוש הפונקציה הזאת אין להשתמש במערך. אין צורך לבדוק שפתיחת הקובץ הצליחה.

```
void set_line_length(const char* file, unsigned int n) {
    FILE* fin = fopen(file, "r");

    unsigned int i=0;
    char ch;

    while((ch = fgetc(fin)) != EOF) {
        if(ch == '\\n')
            ch = ' ';
        putchar(ch);
        if(++i%n == 0)
            putchar('\\n');
    }

    fclose(fin);
}
```

ב. ממשו את שתי הפונקציות הבאות:

- `add_space` – מקבלת כתובת של מחרוזת ומספר שלם וחיובי `n` ומזיזה את המחרוזת `n` בתים קדימה. לדוגמא: עבור המחרוזת "Hello World" והערך `n=2` הפונקציה תשנה את תוכן המחרוזת ל-"HeHello World" בגלל שכל התווים הוזזו 2 בתים קדימה ובמקומות 0 ו-1 נשארו התווים המקוריים.
- `remove_space` – מקבלת כתובת של מחרוזת ומספר שלם וחיובי `n` ומזיזה את המחרוזת `n` בתים אחורה. לדוגמא: עבור המחרוזת "Hello World" והערך `n=2` הפונקציה תשנה את תוכן המחרוזת ל-"llo World"

הניחו ש-`n` תמיד אי שלילי, שהמחרוזת אינה NULL ושהזיכרון בו נמצאת המחרוזת גדול מספיק בשביל ההזזה. בידקו אם `n` גדול מאורך המחרוזת, אם כן צאו מהפונקציה (אבל לא מהתוכנית). לצורך מימוש הפונקציות האלו מותר (אך לא חובה) להשתמש רק ב-`strlen()` מהספרייה הסטנדרטית.

```
void add_spaces(char* p, int n) {
    if(n>strlen(p))
        return;

    for(int i=strlen(p)+1; i>=0; i--)
        p[i+n] = p[i];
}

void remove_spaces(char* p, int n) {
    if(n>strlen(p))
        return;

    while(*(p+n)) {
        *p = *(p+n);
        p++;
    }
    *p = '\0';
}
```

ג. ממשו את הפונקציה `sed_s` שמקבלת שם של קובץ, מחרוזת להחלפה בשם `to_replace` ומחרוזת שתחליף אותה בשם `replacement`, ומדפיסה ל-`standard output` את תוכן הקובץ כך שכל מופע של `to_replace` יוחלף ב-`replacement`. מומלץ להשתמש בפונקציות מהסעיף הקודם. ניתן להניח שהקובץ והמחרוזות תקינים.

למשל: עבור `to_replace="you"`, `replacement="Yoko"` והקובץ `song.txt` שהתוכן שלו מופיע בסעיף א, הפלט יהיה

```
Oh! Darling, please believe me
I'll never do Yoko no harm
Believe me when I tell Yoko
I'll never do Yoko no harm
```

אין צורך לבדוק שפתיחת הקובץ הצליחה. מותר להניח ששורות בקובץ אינן ארוכות מ-256 תווים. לצורך מימוש הפונקציה מותר (אך לא חובה) להשתמש רק בפונקציות הספרייה הסטנדרטית הבאות: `printf`, `strlen`, `fopen`, `fclose`, `fgets`, `strstr`, `strncpy`.

```
void sed_s(const char* file, const char* to_replace, const char* replacement)
{
    FILE* fin = fopen(file, "r");

    char line[257];
    int diff = strlen(replacement) - strlen(to_replace);

    while(fgets(line, 256, fin) != NULL) {
        unsigned int i=0;
        char* p = line;
        while((p = strstr(p, to_replace)) != NULL) {
            if(diff > 0)
                add_spaces(p, diff);
            else
                remove_spaces(p, -diff);
            strncpy(p, replacement, strlen(replacement));
            p += strlen(replacement);
        }
        printf("%s", line);
    }

    fclose(fin);
}
```

שאלה 2

בשאלה זו תממשו מבנה נתונים בשם Table ששומר טבלה של ערכים מאותו טיפוס. הטבלה בנויה מעמודות, כאשר לכל עמודה יש את אותו האורך. ניתן להוסיף עמודות וכן להוסיף שורות לטבלה (כלומר, להגדיל את כל העמודות באותו מספר מקומות).
על מנת לממש את הטבלה השתמשו במודול Array שהממשק שלו נתון. כל עמודה ב-Table תמומש על ידי Array. העמודות תשמרנה במערך של מצביעים.

א. כתבו ממשק (interface) עבור Table על פי העקרונות של Abstract Data Types. הממשק צריך להכיל הצהרות על הפונקציות הבאות (החליטו מה צריכים להיות הפרמטרים והערך המוחזר בהתאם לתיאור של כל פונקציה ושל מבנה הנתונים כפי שתואר למעלה):

- TCreate – יוצרת אובייקט Table
- TDestroy – destructor לאובייקט Table
- TAddColumn – מוסיפה עמודה (אחרי העמודה האחרונה) ל-Table
- TAddRows – מוסיפה n שורות לטבלה
- TGet – מקבלת קואורדינטה x, y (מספר עמודה, x: מספר שורה), מוציאה את האלמנט שבקואורדינטה מהטבלה ומחזירה אותו
- TAt – כמו TGet רק משאירה את האלמנט בטבלה
- TPut – מקבלת קואורדינטה x, y ואלמנט, מכניסה את האלמנט למקום המתאים
- TCount – מקבלת אלמנט ומחזירה את מספר המופעים שלו בטבלה

```
#ifndef ARRAY_H
#define ARRAY_H

typedef struct Array* Array;
typedef void* Element;

// Create an empty array with no cells
Array ArrCreate(Element (*cpy)(Element), void (*fre)(Element));

// destroy an array
void ArrDestroy(Array arr);

// Get Array size
unsigned int ArrSize(Array arr);

// Change the size of the Array
void ArrResize(Array arr, unsigned int new_size);

// Insert an element to position pos in arr
Element ArrPut(Array arr, unsigned int pos, Element e);

// Get the element in position pos but leave it in the Array
Element ArrAt(Array arr, unsigned int pos);

// Get the element in position pos and remove it from the Array
Element ArrGet(Array arr, unsigned int pos);

#endif // ARRAY_H
```

```

#ifndef TABLE_H
#define TABLE_H

typedef struct Table* Table;
typedef void* Element;

Table TCreate(Element (*cpy)(Element), void (*fre)(Element),
              bool (*cmp)(Element, Element));
void TDestroy(Table t);
void TAddColumn(Table t);
void TAddRows(Table t, unsigned int n rows);
Element TGet(Table t, unsigned int x, unsigned int y);
Element TAt(Table t, unsigned int x, unsigned int y);
Element TPut(Table t, unsigned int x, unsigned int y, Element e);
unsigned int TCount(Table t, Element e);

#endif

```

ב. ממשו את TCreate ואת ה-struct ששומר את המידע של הטבלה. אין צורך לבדוק הצלחה של הקצאת זכרון

```

struct Table {
    Array* columns;
    unsigned int num_columns;
    Element (*cpy)(Element);
    void (*fre)(Element);
    bool (*cmp)(Element, Element)
};

Table TCreate(Element (*cpy)(Element), void (*fre)(Element),
              bool (*cmp)(Element, Element)) {
    Table t = (Table)malloc(sizeof(struct Table));
    t->columns = NULL;
    t->num_columns = 0;
    t->cpy = cpy;
    t->fre = fre;
    t->cmp = cmp;

    return t;
}

```

ג. ממשו את הפונקציה TCount

```

unsigned int TCount(Table t, Element e) {
    unsigned int n = 0;
    for(unsigned int y=0; y<t->num_columns; y++) {
        for(unsigned int x=0; x<t->columns[y].size(); x++) {
            n += (t->cmp(e, t->columns[y].at(x)));
        }
    }
    return n;
}

```

שאלה 3

א. כתבו פונקציה בשם `count_ones` שמקבלת מספר מטיפוס `unsigned int` ומחזירה את מספר הביטים שבו שהם 1. למשל: עבור 1285 (00000000 00000000 00000101 00000101) הפונקציה תחזיר 4. חתימת הפונקציה:

```
unsigned int count_ones(unsigned int n)

unsigned int count_ones(unsigned int n) {
    unsigned int total = 0;
    for(unsigned int i=0; i<sizeof(unsigned int)*4; i++) {
        total += ((1 << i) & n) != 0;
    }
    return total;
}
```

ב. ממשו פונקציה שמקבלת מספר שלם `n` ומחזירה את המספר האי-זוגי הגבוה ביותר אותו ניתן ליצור מהביטים של `n`. אם כל הביטים של `n` הם 0 הפונקציה תחזיר 0. ניתן להשתמש בפונקציה מהסעיף הקודם. חתימת הפונקציה:

```
unsigned int create_largest(unsigned int n)

// Idea: create a number with 1 at the lsb and the rest of the 1's at
// the msb's
unsigned int create_largest(unsigned int n) {
    unsigned int total_1s = count_ones(n);
    if(total_1s == 0)
        return 0;

    // num will keep the number itself, must have 1 in its lsb
    // add will keep the next bit to turn on - starting with the msb
    // and going downwards
    unsigned int num = 1, add = 1 << sizeof(unsigned int)-1;
    total_1s -= 1; // We used one 1 already

    for(unsigned int i=0; i < total_1s; i++) {
        num += add;    // same as num |= add
        add >>= 1;
    }
    return total;
}
```

ג. כתבו פונקציה בשם turn_on_bits שמקבלת מצביע למערך של unsigned char, אינדקס של ביט pos ומספר n, ומדליקה את n הביטים החל מהביט ה-pos במערך. למשל: עבור pos=23 ו-n=5, הפונקציה תדליק את הביט האחרון (MSB) בבית השלישי ואת ארבעת הביטים הראשונים (LSB) בבית הרביעי. הניחו שהביטים שצריך להדליק נמצאים במקום חוקי במערך.

חתימת הפונקציה:

```
void turn_on_bits(unsigned char* arr, unsigned int pos, unsigned int n)

void turn_on_bits(unsigned char* arr, unsigned int pos, unsigned int n) {
    unsigned int byte = pos/8;
    unsigned int bit = pos%8;
    for(; n > 0; n--) {
        arr[byte] |= (1 << bit);
        bit++;
        if(bit == 8) {
            bit = 0;
            byte++;
        }
    }
}
```

שאלה 4

א. נתון הקובץ roxanne.txt שמכיל את מילות השיר Roxanne של להקת פוליס. הניחו שהמילים בכל שורה מופרדות על ידי רווחים. כיתבו שורות פקודה שמבצעות את הדברים הבאים על הקובץ:

1. מדפיסה את מספר השורות שמכילות את המילה Roxanne (בצורה הזאת, עם R בתחילת שורה)

2. מדפיסה את מספר הפעמים שבהן המילה You מופיעה בתחילת השורה

3. מדפיסה את מספר הפעמים שבהן המילה you ללא הבדל uppercase/lowercase מופיעות בשיר. שימו לב שלא מספיק לספור כאן את מספר השורות כי המילה יכולה להופיע מספר פעמים בשיר

4. מדפיסה את מספר המופעים של כל תו בראש שורה, עם הבדל של אותיות קטנות/גדולות. אין צורך למיין את התווים לפי מספר המופעים

1. `grep roxanne.txt | wc -l`
2. `cut -f1 -d" " roxanne.txt | grep Roxanne | wc -l`
3. `cat roxanne.txt | tr " " "\n" | grep -i you | wc -l`
4. `cut -b1 roxanne.txt | sort | uniq -c`

ב. כיתבו סקריפט שמקבל מספר מהמשתמשת ומדפיס, בשורות נפרדות, את המספר ואת סכום כל המספרים עד לאותו מספר, מופרדים בטאבים. הניחו שהסקריפט קיבל מספר שלם אבל בידקו שהמספר חיובי. למשל: עבור 5 הסקריפט ידפיס

```
1      1
2      3
3      6
4     10
5     15
```

```
#!/bin/bash

if [[ $# -ne 1 || $1 -le 0 ]]; then
    echo -e "\nUsage: $0 <n>\n"
    exit
fi
n=$1
t=0
for i in $(seq 1 $n) ; do
    t=$((t+i))
    echo -e "$i\t$t"
done
```


ג. כתבו סקריפט שעובר על כל קבצי הטקסט בתיקייה שמתקבלת כפרמטר (בידקו שהתקבל פרמטר ושהתיקייה היא אכן תיקייה) ומדפיס עבור על אחד מהקבצים והתיקיות בה את המידע הבא, מופרד בטאבים:

עבור תיקיות:

- שם התיקייה

- המילה "dir"

- מספר הקבצים+תיקיות בתוכה

עבור קבצים שלא מסתיימים ב-txt:

- שם הקובץ

- המשפט "non-txt file"

עבור קבצים שמסתיימים ב-txt:

- שם הקובץ

- המשפט "txt file"

- מספר השורות בקובץ

- מספר התווים שאינם רווחים, טאבים או תווי ירידת שורה

```
#!/bin/bash
```

```
if [[ $# -ne 1 || ! -d $1 ]] ; then
    echo -e "\nUsage: $0 <dir-name>\n"
    exit
fi
```

```
dir="$1"
```

```
for item in $dir/* ; do
    if [[ -d $item ]] ; then
        echo -e "$item\tdir\t$(ls $item | wc -l)"
    else
        if [[ $(echo $item | grep "txt$" | wc -l) -eq 0 ]] ; then
            echo -e "$item\tnon-txt file"
        else
            nchars=$(cat $item | sed 's/[ \t]//g' | wc -c)
            nlines=$(cat $item | wc -l)
            echo -e "$item\ttxt file\t$((nchars-nlines))"
        fi
    fi
done
```